淘天营销中后台生码工作流最佳实践

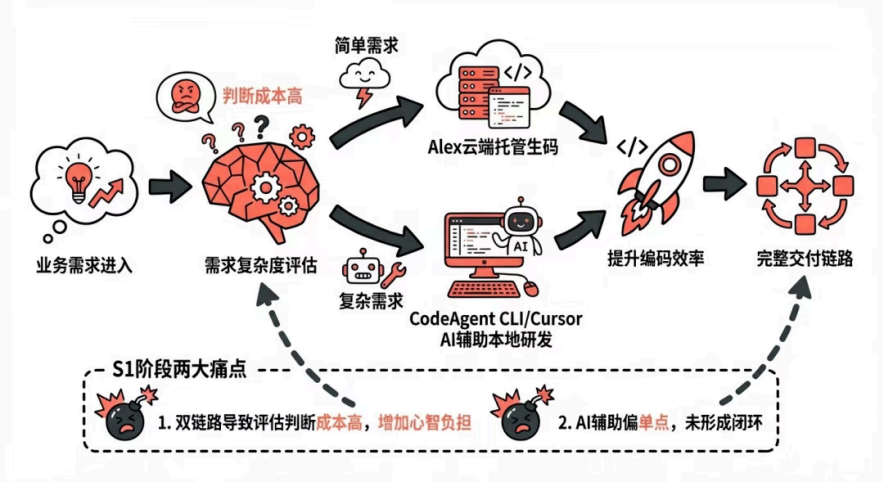
原创 营销前台技术团队 大淘宝技术 2026年4月27日 01:16 浙江 203人

本文系统总结了营销中后台在财年初推进AI生码提效的最佳实践升级路径：统一收敛至云端托管生码（基于AoneSuper），解决本地研发环境不一致、AK管理难、执行易中断等问题；1.构建跨仓库工作区（git submodule + turborepo）支持多仓协同；2.打造可编排场景化工作流，覆盖需求理解→编码→构建发布全链路；针对迁移/重构（高确定性）采用架构说明文档+领域Skill固化规则；针对日常迭代（低确定性）引入功能树实现精准查表式知识供给，并通过D2C/API还原优化、知识自动沉淀形成提效飞轮。核心方法论：给恰好够用的精确知识、确定性逻辑交工程、知识建正向循环。

01 生码提效路径的重新梳理

营销中后台在财年初，面向业务研发的简单需求和复杂需求探索出了两条提效路径：对于**简单需求**，可以从aone工作项出发，直接在云端Alex平台完成迭代创建和编码研发，**一站式托管生码**。对于**复杂需求**，考虑到云端不能一次性完成所有研发任务，人工二次干预成本较高，降级到用本地Cursor/CodeAgent CLI工具，配合业务自定义规则**辅助生码**。

实际需求研发被分割成了两条路径，天然增加了业务开发的评估判断成本。同时，AI辅助提效只覆盖到了几个节点（核心是编码研发），在完整的需求交付生命周期中，还需要大量的人力来做流程串联，才能完成最终需求交付。



从上述两个痛点问题出发，决定收敛两条AI生码提效路径，扩大AI生码可落地的场景范围，提升AI生码的实际效果。

02 本地还是云端？路径收敛的抉择

既然要收敛，就要在“AI 辅助本地研发”和“云端托管生码”之间做出选择。这应该是所有AI Coding平台都会面临的选择，先说结论：我们选择了统一收敛至云端托管生码。

为什么不选本地

本地研发模式在团队实际推行中，遇到了以下三个实际问题：

环境配置难以统一，问题排查协作成本高：不同同学的本地环境（系统版本、Node 版本、网络代理等）差异巨大，同一套插件和 MCP 配置在部分同学环境中频繁出问题；当某位同学本地 AI 工具出现异常时，由于环境差异，其他人很难复现问题，远程协助排查效率低，往往只能“到工位上看一眼”。

生态用工的 AK 管理困难：团队中有不少生态用工同学，本地研发模式下 AK 需明文存储在个人设备上，分发、轮换、回收缺乏统一管控机制，安全隐患和管理成本并存。

执行易中断：本地执行需要依赖电脑持续在线，且网络链接稳定，经常遇到一个长任务在执行过程中，电脑自动息屏或网络断开，导致任务中断，需要手动触发继续执行，增加了任务执行的总耗时，且需要时刻关注任务状态，无法专注做其他工作。

借力集团已有AI基建

虽然云端托管生码可以解决上述本地研发的实际问题，但其也存在一些固有劣势，需要投入大量精力去做打磨，才能搭建出一套好用、可靠的云端生码环境。这里我们选择借力集团已有AI

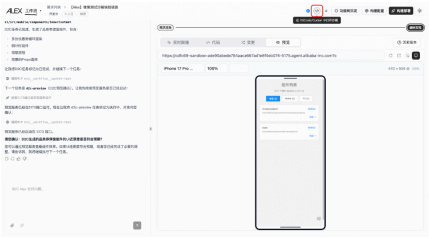
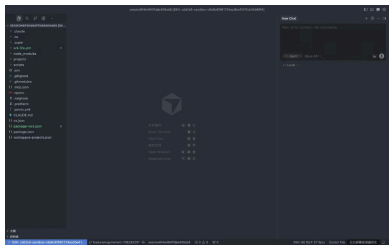
基建，将底层云端环境切换至AoneSuper（集团内提供云端生码沙箱环境的平台），在此基础之上结合业务特性，来做工程能力定制，和AI生码效果优化。

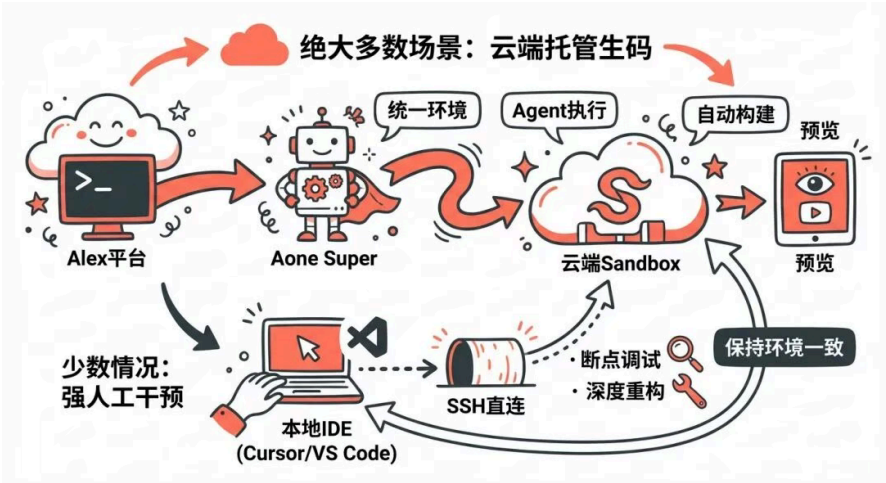
为什么选择 Aone Super 而非继续自建？

- S1 自建基于 LangGraph 的多 Agent 架构，基建维护成本高，CodeAgent CLI 社区生态迅速成熟（Skills、MCP、SubAgent 等原生能力），在生码场景自建LangGraph方案边际收益递减。
- 接入 Aone Super，将投入重心从基建打磨转向业务效果优化。

收敛后的云端一体化研发流程

在AoneSuper底层基建的加持下，可以支持本地IDE直连远程沙箱，实现了云端一体化的研发流程：

云端生码	本地IDE直连沙箱
绝大多数场景可云端执行全流程	少数场景较复杂，需要强人工干预，可本地IDE直连沙箱，具备原生的开发体验
	



03 云端托管生码的配套工程能力建设

云端路径确定后，工程能力还需要跟上预期的两个核心目标，要**扩大可落地场景范围**，就必须解决日常大量存在的跨仓库研发诉求；要**让 AI 辅助辐射需求交付全链路**，就需要一套可编排

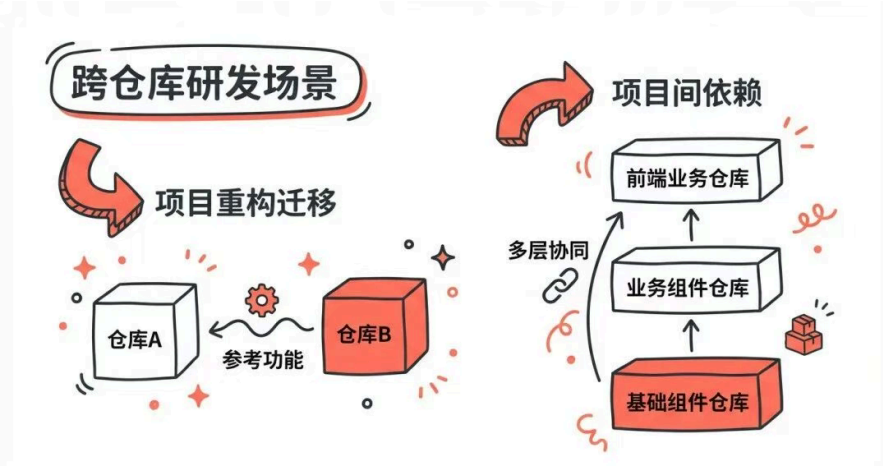
的场景化工作流把各环节串联起来。

支持跨仓库研发的工作区

- 设计背景

营销中后台日常需求中有大量跨仓库研发场景，可归纳为两类：

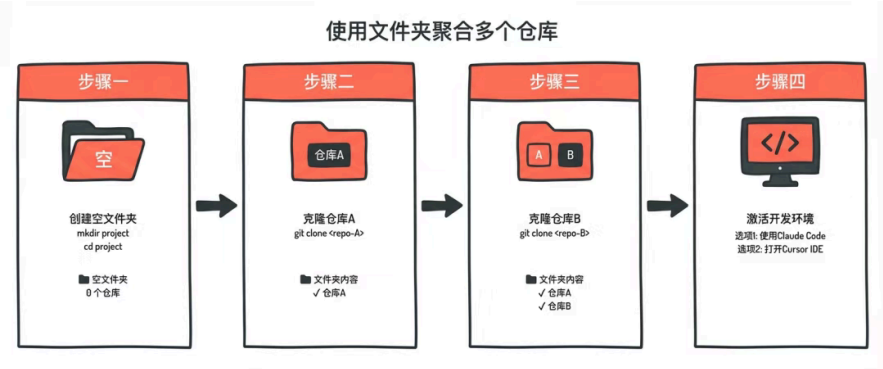
- **项目重构/迁移：** 仓库A需要参考仓库B的部分功能实现；
- **项目间相互依赖：** 前端业务仓库 <-- 业务组件仓库 <-- 基础组件仓库，多层协同，多层协同。



- 实际方案设计

聚合需求相关的仓库

基于社区方案的思路，最简单的方式，可以通过创建一个“空文件夹”，在这个文件夹中聚合需求相关的所有仓库，即可实现跨仓库代码感知。

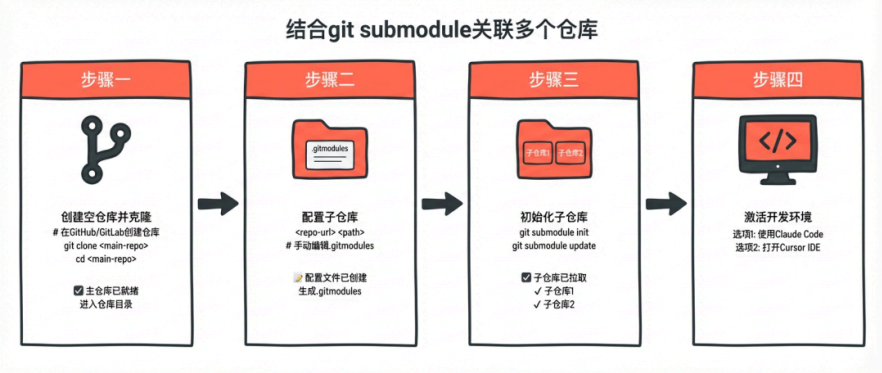


引入git submodule

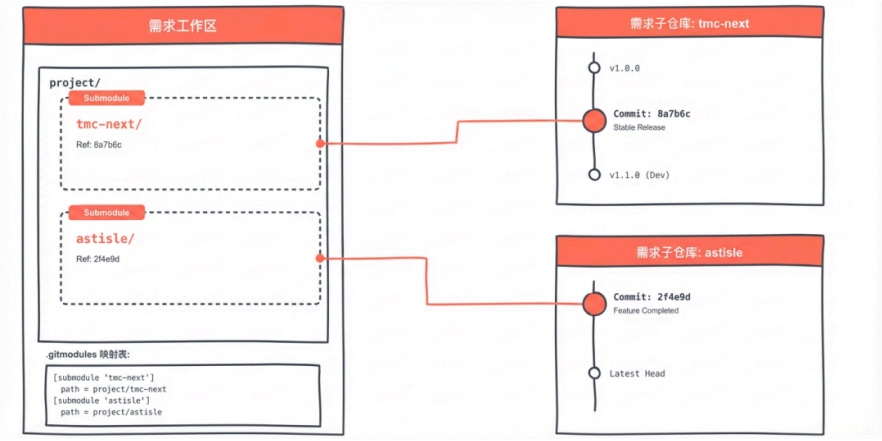
但在实际生码场景下，还需要在“空文件夹”下存放**Agent消费的配置项**（Skills、MCP、SubAgent等），如果这个“空文件夹”脱离了git体系，很难维护管理这些配置项。此外，在复杂

需求场景下，生码过程中可能会生成**一些中间产物**（需求理解、方案设计、任务列表等）用来聚焦Agent执行任务的注意力，类spec-kit的思想，这些中间产物不适合离散的存放“空文件夹”下的业务仓库，更适合存放在“空文件夹”下。

因此，在用“文件夹”聚合需求仓库的基础上，借助git submodule的能力，将外层的“文件夹”作为一个独立的git空间，使其具备git托管的能力。同时，在其内部可以关联已有业务仓库git空间，通过“软链接”将多个业务仓库聚合到一起。



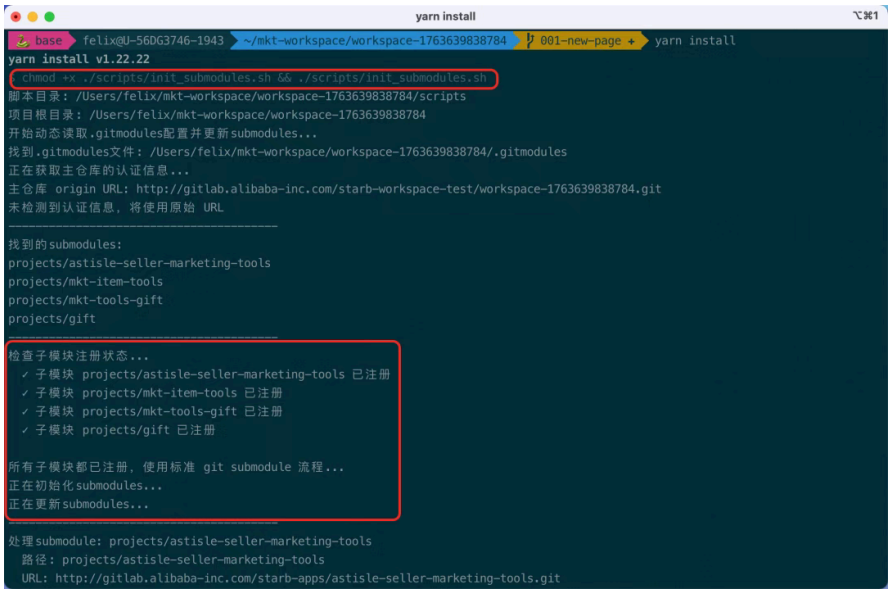
需求交付过程中，可以仅面向需求工作区进行研发（聚合多个仓库，无需再多个仓库来回切换开发），对子仓库的代码修改会提交到对应子仓库的git空间，不改变业务代码的维护方式：



需求工作区和需求子仓库关联关系

消除副作用

git submodule的引入，解决了跨仓库代码感知的能力，但其也带来了一系列新的git命令（以往不会使用到），为了降低业务研发同学对这一层的理解成本，通过实现一些脚本来自动化执行submodule操作（仓库初始化、代码拉取/提交等）：



研发调试优化

对于**项目间相互依赖**（前端业务仓库 <-- 业务组件仓库 <-- 基础组件仓库）的跨仓库研发场景，还有一个比较头疼的问题是：如何在研发调试过程中，修改完基础组件或业务组件的代码后，前端业务仓库能实时应用到修改后的代码，并在预览页面中体现出来。以往这种场景下，需要为每一层依赖手动配置link，过程中还会因为环境等因素，导致link较难生效。因此，考虑对工作区方案做进一步优化，尝试来提升这类场景的研发调试体验。

上面的完整工作区结构设计，在忽略git submodule这一层后，不难发现和传统的monorepo项目几乎一致，外层的“文件夹”对应monorepo的主仓库，内部聚合的业务需求仓库对应monorepo的一个个子包。因此，可以借助monorepo相关构建工具（turborepo、nx等），来进行优化。

拿turborepo工具举例，可以通过以下配置实现一键启动所有需求子仓库的服务，同时自动配置子仓库间的依赖link：

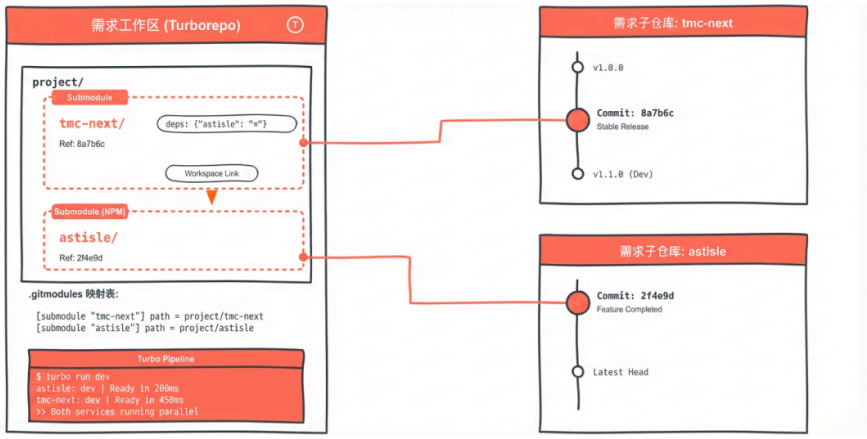
```
{
  "workspaces": [
    "projects/*"
  ],
  "scripts": {
    "build": "turbo run build",
    "lint": "turbo run lint",
    "test": "turbo run test",
    "clean": "turbo run clean",
    "start:workspace": "turbo run start:workspace"
  },
  "devDependencies": {
    "turbo": "latest"
  }
}

```

```
{
  "$schema": "https://turbo.build/schema.json",
  "globalDependencies": [
    ".env",
    "tsconfig.json"
  ],
  "globalEnv": [],
  "daemon": true,
  "concurrency": 3,
  "pipeline": {
    "build": {
      "dependsOn": ["^build"],
      "outputs": [
        "build/**",
        "dist/**",
        ".next/**",
        "!.next/cache/**"
      ],
      "inputs": [
        "**/*",
        "!**/?(*.)+(spec|test).[jt]s?(x)?(.snap)",
        "!tsconfig.spec.json",
        "!.eslintrc.json",
        "!eslint.config.js"
      ],
      "cache": true
    },
    "lint": {
      "outputs": [],
      "cache": true
    },
    "test": {
      "outputs": [],
      "cache": true
    },
    "docs:build": {
      "outputs": [],
      "cache": true
    },
    "start:workspace": {
      "dependsOn": ["^build"],
      "cache": false,
      "persistent": true
    },
    "clean": {
      "cache": false
```

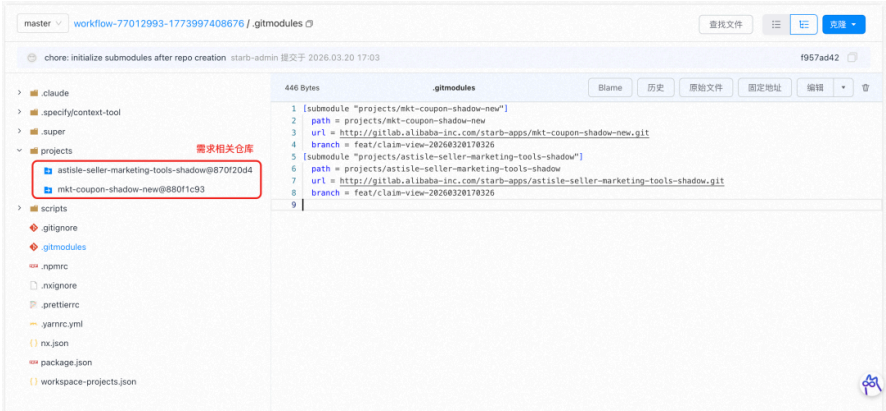
```
    }
  }
}

```



项目间相互依赖研发调试优化方式

完整的需求研发工作区结构如下，其本身只是工程能力上的建设，因此不限制是在云端生码平台使用，还是本地AI辅助研发使用，把其当作一个正常monorepo仓库即可，projects目录会通过git submodule聚合需求相关的所有子仓库：



可编排的场景化工作流

- 设计背景

S1 阶段 AI 辅助的介入以编码研发节点为核心，但完整的需求交付链路远不止于此——编码前有需求理解、方案设计、任务拆解，编码后有构建、发布、验收等环节，这些仍依赖大量人工串联。AI 只优化了中间一段，整体交付效率的天花板被前后两端卡住。S2 的核心目标是将 AI 辅助从编码节点扩展到需求交付全链路，通过可编排的工作流将前置准备、编码研发、后置构建部署串联为一体，减少人工流程串联成本。

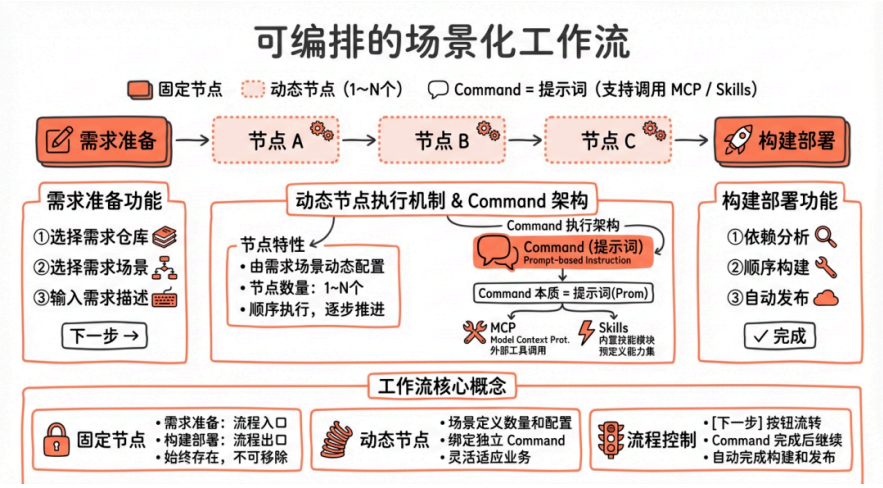
这一思路受到社区 spec-kit 实践的启发，spec-kit 的核心理念是在执行前先生成结构化 Spec（规格说明），将模糊的需求描述转化为可逐步执行的任务清单，让 Agent 执行每一步时都有明确的上下文和验收标准，从而大幅降低长任务的失控风险。

然而，营销中后台各业务场景差异显著——迁移重构、日常迭代、缺陷修复的节点序列、所需工具、Prompt 模板差异巨大，无法用同一套固定流程覆盖。因此，在通用工作流框架之上，需要提供场景级定制能力：每个场景可以独立配置动态节点序列、绑定专属 Command 和 Skills，做到框架统一、场景差异化。

- 工作流整体架构

整体设计以 **CodeAgent CLI 的 Command 机制**作为工作流节点的推进方式，每次触发时将文件内容作为 Prompt 注入当前对话，天然支持在 Prompt 中声明调用 MCP 工具和 Skills。我们将每个交付节点的任务意图写成一个 Command，由平台在适当时机依次触发，从而将 CodeAgent CLI 的单次对话能力串联为一条可编排的多节点工作流。

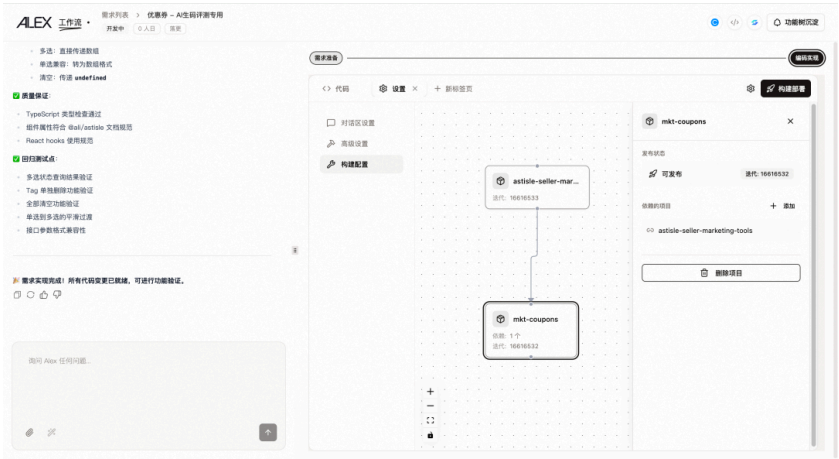
工作流由**固定节点**和**动态节点**两类构成，动态节点的数量与内容由需求场景定义，整体执行顺序为：需求准备 → 动态节点（1~N 个）→ 构建部署：



- **固定节点 – 需求准备**（入口）：选择需求仓库（支持多仓库配置）、选择需求场景（决定动态节点数量）、输入需求描述（支持场景专属模板）。

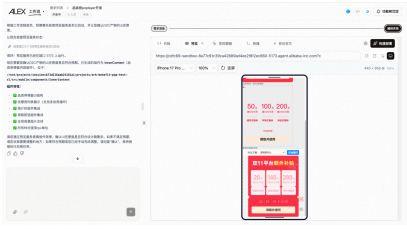


- **动态节点**（1–N 个，由场景定义）：每个节点绑定独立的 Command。Command 本质是一份 Prompt，定义该节点需要执行的任务和行为；Prompt 中可声明调用 MCP（外部工具，如 aone-km 读取需求文档、d2c还原等）和 Skills（内置技能模块，将营销中后台各垂直子域的生码最佳实践固化为可复用 Skill，在节点执行时按需调用，补充私域研发知识），节点顺序执行，逐步推进。
- **固定节点 – 构建部署**（出口）：自动梳理仓库间依赖关系 → 按依赖顺序执行构建 → 依序完成所有仓库发布。



- 落地工作流示例

基于当前可编排的场景化工作流，业务可灵活定制动态节点的同时，可快速复用需求准备和一键构建能力，以下为几个典型场景的实际落地工作流示例：

ToB丨需求迭代丨商家营销工具	ToC丨需求迭代丨POP弹窗
	
ToB丨技术重构丨营销资金设置端迁移	ToB丨技术重构丨商家营销工具N合一迁移

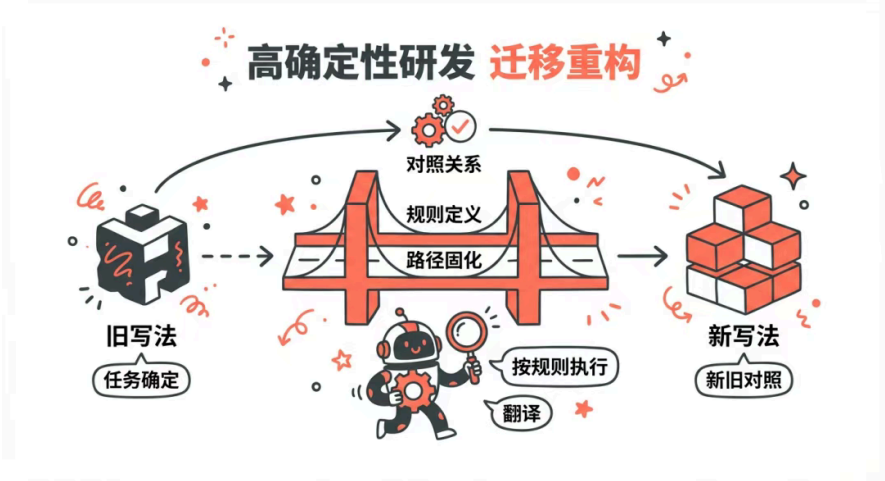


04 两类研发任务的差异化生码优化策略

营销中后台的日常研发任务有两类：迁移/重构（长尾架构问题治理）、日常迭代（常规业务需求），在实际落地过程中，发现这两类研发任务需要的优化策略截然不同，通过不断试错，摸索出了两类研发任务的差异化生码优化策略。

高确定性研发任务：迁移 / 重构

特点：任务确定性高，迁移规范可前置梳理，执行路径可完全固化。迁移重构有明确的"新旧对照关系"——每一类旧写法对应固定的新写法，这套规则可以在迁移启动前被完整定义，Agent 的核心工作是**按规则执行翻译。**

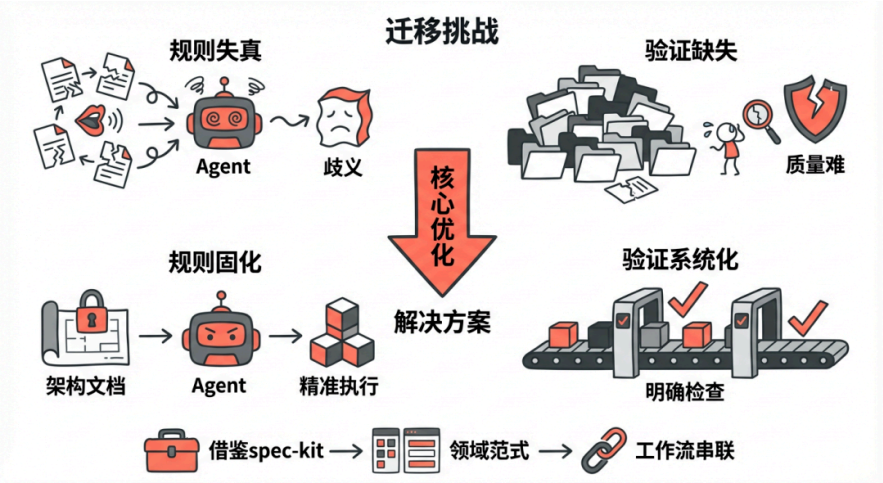


- 核心挑战及解决思路

迁移重构场景的最大挑战不是“Agent 不够聪明”，而是**规则传递失真**：迁移规范通常散落在设计文档、口头约定和历史代码中，Agent 接收到的是碎片化、歧义多的描述，执行过程中自行补全细节，导致输出偏离预期。另一个挑战是**执行验证缺失**：迁移任务往往跨越大量文件，人工逐一 review 成本极高，质量保障困难。

因此，迁移场景的优化核心在于两点：**将规则前置固化**（让 Agent 执行生码时不需要再推理规则）；**将验证系统化**（让每个关键迁移点都有明确的检查项）。整体借鉴spec-kit的思路，

为每个迁移重构场景前置设计架构说明文档，配合营销中后台标准化的表格、表单领域范式，再通过定制场景化工作流实现完整串联。



以下拿**「商家营销工具N合一迁移」**场景举例，介绍迁移 / 重构研发任务的生码优化策略：

- 迁移规则固化：架构说明文档

借鉴 spec-kit「先规格后执行」的理念，在前期先由业务研发同学完成**架构说明文档**的梳理——这份文档作为 Agent 执行的唯一规则源，在每个执行节点的 Command 中优先加载。设计原则是**穷举所有迁移决策**，覆盖以下核心内容：

新旧架构对照：明确描述旧架构和新架构的核心差异，规定哪些旧写法对应哪些新写法，不留歧义空间。以 N 合一迁移场景为例，架构说明明确规定：旧架构（声明式配置）的每一类配置项，对应新架构（纯源码组件）下具体的实现模式；组件按原子化设计原则拆分粒度，每个功能单元独立目录；组件层仅负责渲染，业务逻辑统一内聚至 hooks 层。

禁止行为清单：明确列出迁移过程中不能做的事，以强制规则形式写入，而非建议。如“禁止在组件层写业务逻辑”、“禁止引入非规范组件库”等。

接口映射规则：当旧接口与新接口字段不一致时，预先定义字段映射关系，Agent 按映射表转换，无需自行推断。

- 领域知识编码：Skill 作为按需手册

营销中后台的迁移场景高度集中在两类：**表格**（列表页）和**表单**（创建/编辑页）。这两类场景各自有一套完整的开发范式，内容详尽，但如果在每次迁移任务中都把两份完整范式同时灌入 Agent，不仅大量内容与当前迁移无关，还会稀释真正重要的规则，形成新的信息过载。

借助 **Skill 按需加载**机制，将两类场景的开发规范分别封装，在编码节点根据实际迁移内容有 Agent选择性加载，避免无关知识进入上下文：

表单开发 Skill（ProForm）：规范基于 `ProForm + useForm` 的表单开发范式，核心解决三类问题：表单项该用哪种模式实现、显隐联动该用哪种写法、全局数据和表单数据如何隔离管理。

表格开发 Skill（ProTable）：规范基于 `ProTable + useTable` 的表格开发范式，核心解决两类问题：数据查询链路的标准写法、搜索区与表格列的配置规范。

- 工作流节点化：场景化执行链路

借助前文的场景化工作流框架，配置场景化工作流，把各执行环节完整串联：

了解旧架构：在编码启动前将旧架构的相关仓库加载进工作区，让 Agent 可以直接读取旧实现作为迁移参考，确保在动手前对迁移源有充分了解。

计划制定阶段：Agent 依次完成规格撰写（功能拆解与技术方案）、歧义澄清（识别需求描述中的不确定项）、迁移计划生成（需要创建/修改的文件清单、改动意图、组件映射关系）、任务拆解、以及跨文档一致性分析。在该阶段执行中，有问题可直接通过对话进行纠偏，而非等代码生成后返工。

编码实现阶段：严格按计划逐文件生成迁移代码，计划文档已明确每个文件的改动意图，Agent 无需在编码阶段做架构判断。

验证收尾阶段：针对迁移结果进行系统化验收，表单迁移场景逐字段生成测试用例，对有联动关系的字段自动生成状态流转图；通用重构场景则生成验收检查清单，逐项核查迁移完整性。



代码生成前问题 → 对话纠偏，无需返工

- 落地效果

- 商家营销工具N合一迁移：以赠品重构场景为例，功能点完成率 71.43%，TC 通过率 66.7%，TC 通过 + 部分通过率 79.1%，AI 编码带来的效率提升 58.13%。
- 营销资金设置端迁移：完成21个页面迁移（主要为外包同学进行迁移），整体迁移时间降低 62.73%。

之所以能取得这样的效果，根本原因在于迁移场景的规则集有限且可穷举：旧架构写法固定、新架构目标写法固定，所有决策点可在启动前识别和记录，架构说明文档理论上可覆盖全部迁

移场景，整体的确定性较高。

低确定性研发任务：日常迭代

特点：任务确定性低，需求方向由产品随业务发展决定，无法提前穷举；改动范围跨功能模块，Agent 需要在运行时理解架构、定位入口、正确调用组件 API，任何一步出偏差都会导致生码质量大幅下滑。



- 核心挑战及解决思路

迁移重构方案成立的前提是“规则可穷举”——旧写法有限、新写法固定，架构说明文档可以覆盖所有场景。但日常迭代打破了这个前提：**需求由产品随业务发展决定，功能点的组合无法预先穷举，每个需求涉及的改动无法预知。**

该场景落地初期，沿用了迁移重构场景的思路，用一份大而全的架构规范文档 + 多个领域Skills来覆盖所有日常迭代场景，但效果远低于预期，暴露出系统性缺陷：

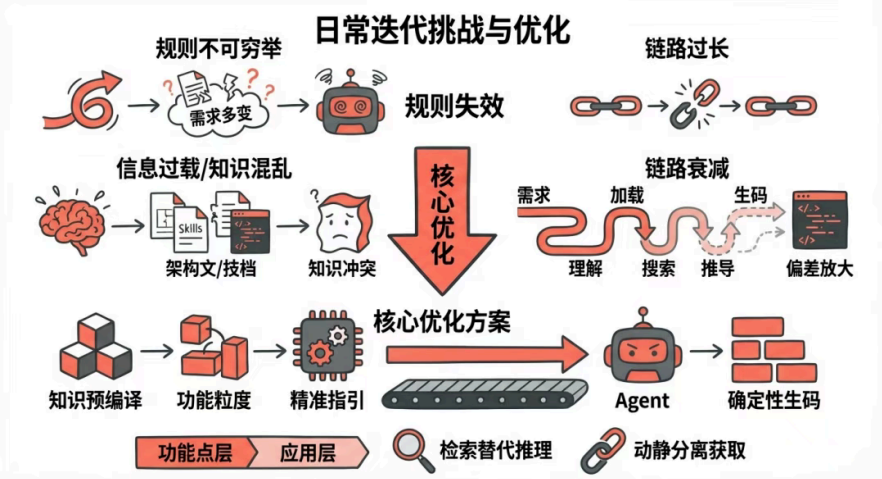
信息过载：单次生码任务注入过多的上下文知识，关键信息容易被低价值信息稀释，实际起作用的知识密度很低。

知识来源多元、优先级不明确：架构规范、大而全的领域Skills、CodeBase 检索结果同时注入上下文，各来源对同一问题的描述存在重叠甚至冲突（如 Formily 用法在架构规范和 Skill 中均有描述），Agent 无法判断以谁为准，容易产生幻觉。

链路过长、衰减严重：需求输入 → 理解架构规范 → 加载 Skills → 搜索代码位置 → 推导方案 → 生码，6+ 步链路，每步都可能引入噪声，最终输出与预期的偏差随链路长度指数级放大。

核心问题在于：**给 Agent 的是通用、抽象的知识，但生码需要的是具体、精确的信息。**优化方向不是“给更多知识”，而是“给恰好够用的精确知识”。

因此，日常迭代场景的优化核心在于两点：**将知识预编译**（把通用架构知识转化为功能点粒度的精确信息，让 Agent 生码前可以先精确召回本次需求需要的研发指引）；**将获取动静分离**（按需求点一次性召回静态研发指引，生码过程中再按需动态查询组件、 API 等知识，避免无关信息占用上下文）。整体基于「产品功能树」的思路，为每个业务应用预先梳理功能点与代码位置的映射关系，配合分层研发指引（功能点层 → 应用层）沉淀精细化知识，以检索替代推理，实现确定性更高的生码效果。



以下拿「商家营销工具日常迭代」场景举例，介绍日常迭代研发任务的生码优化策略：

- 引入功能树：以查表替代推理

在前期一股脑丢改Agent足够多的知识受挫后，尝试大道至简，发现只给Agent当前需要实现功能的代码入口，结合它自身的CodeBase检索，也能比之前的生码效果好，于是摸着这个“石头”进一步引入了产品功能树的能力。功能树是日常迭代生码优化的核心载体，它将每个业务应用的所有功能点预先整理成一棵树状结构：

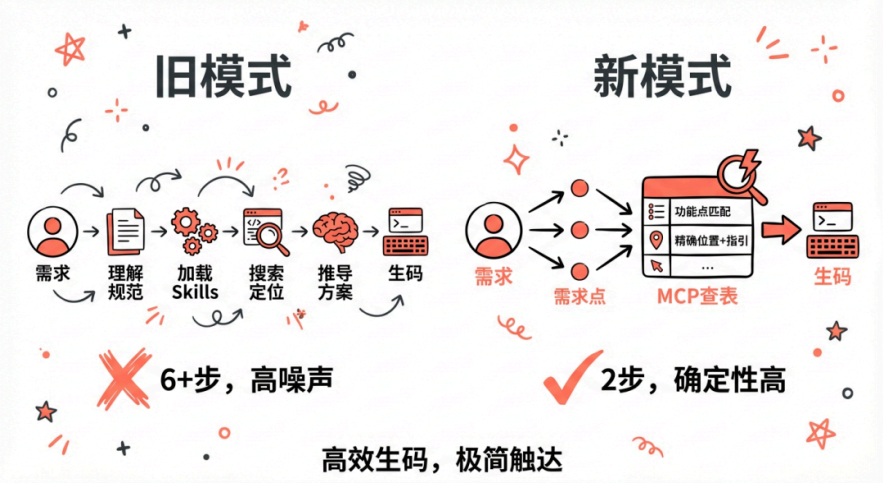
```
└─ 优惠券
    ├─ 首页
    │   ├─ 首页_面包屑
    │   ├─ 首页_公告
    │   ├─ Banner
    │   │   └─ 推荐设置入口
    │   ├─ Tab切换
    │   │   └─ Tab拓展区
    │   │       ├─ 批量创建
    │   │       └─ 存储版数量查看
    │   ├─ 券管理列表
    │   │   ├─ 券管理列表_筛选表单
    │   │   ├─ 券管理列表_单元格展示
    │   │   └─ 券管理列表_列表操作
```

```
15      |   ├─ 查看活动数据
16      |   └─ 创建页入口列表
```

每个叶节点对应一个具体的功能区域（如筛选区、列配置、表格扩展区），并与四类资产显式绑定：

- 关联代码：文件路径 + 文件概要，告诉 Agent 改哪个文件、这个文件的职责是什么；
- 关联接口（可选）：接口描述、接口路径、请求方法、出入参定义，告诉 Agent 数据从哪来、格式是什么、怎么使用；
- 关联设计稿（可选）：设计稿 ID（接入淘天 D2C 能力）+ 截图，告诉 Agent 改成什么样；
- 关联 Skill（可选）：原子化的 Skill，如何新增表单项、如何实现表单校验等，告诉 Agent 按什么规范和模式去改。

生码之前先对原始需求进行需求系分，拆解为多个需求点，每个需求点独立执行生码逻辑，在生码前 Agent 通过 alex-code-knowledge MCP 将当前需求点与功能树节点进行匹配，若匹配到，直接返回精确的代码位置和研发指引，整个“理解架构 → 搜索代码 → 推导方案”的推理链路被压缩为一次查表操作。

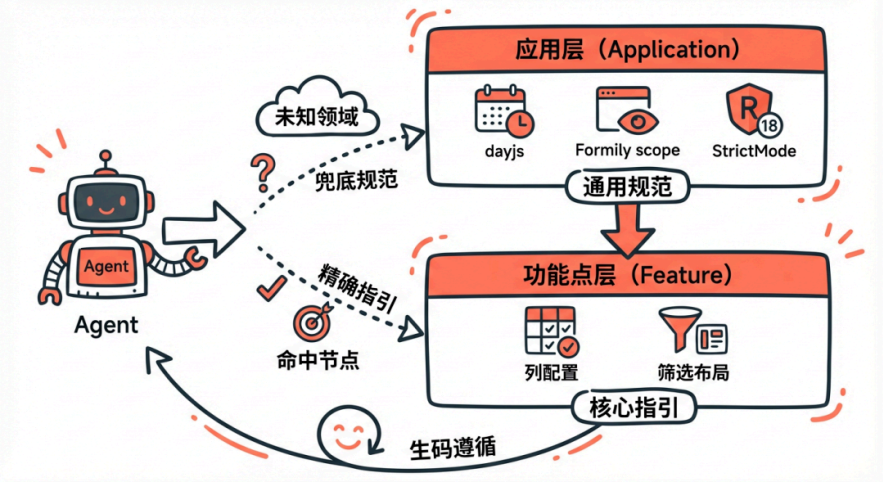


- 研发指引的分层与按需获取

功能树解决了“匹配到功能点时，如何给精确知识”的问题，还需要在此基础之上设计分层研发指引，来保证可以覆盖更多场景。

分层：提上限与保下限

功能树只能覆盖当前已有的功能，无法预知未来产品新增的功能，因此需要设计分层的研发指引，把和功能点不强相关的部分放在应用层。

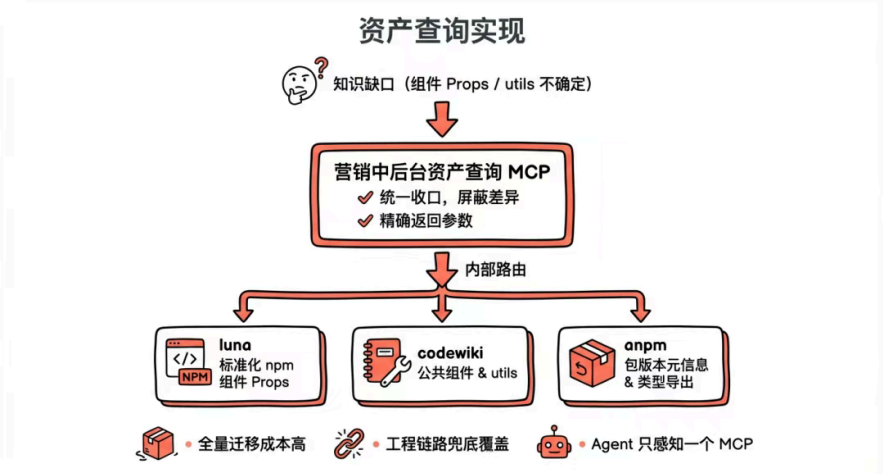


当需求点匹配到了功能树中已有的节点时，Agent 可以获得高度精确的指引。当需求点涉及全新功能或功能树尚未覆盖时，应用层的通用规范承担了兜底研发指引的职责，确保 Agent 即便在“未知领域”生码，也能遵循当前应用的技术选型和开发规范，提上限、保下限。

按需：编码中动态查组件

即便静态指引覆盖了大部分场景，生码过程中仍会遇到“这个组件具体支持哪些 Props”这类组件 API 层面的知识缺口。集团内的luna资产中心可以解决这类问题，但由于营销中后台公共组件、utils较多，全量迁移至luna成本较高，因此先通过一个资产使用指引Skill来包装各种资产的查询方式，内部调用各个平台（luna、codewiki、anpm）的开放接口，但实际执行过程中发现Agent的指令遵循度较差，常常弄错当前npm包应该调用哪个开放接口获取。

后面思路做了转变，这个Skill内部的逻辑本质就是一堆switch case，如果把这堆死板的路由逻辑直接丢给Agent，相当于让一位米其林主厨不去专注火候与调味，而是被按在仓库里靠翻纸质台账核对配料编号——不仅大材小用，更会严重干扰他的核心创作节奏。改变后，我们通过包装统一的营销中后台资产查询MCP，将组件和Utils的参数获取收敛到工程链路中，Agent只需按需调用，即可动态获取精确上下文。这也成为了我们后续的最佳实践：确定性逻辑交还工程，不确定性决策留给Agent，真正做到“合适的角色做合适的事”。



- D2C & API 还原效果优化

日常迭代需求中，经常包含设计稿（D2C）和接口（API）的引用，这两类输入在早期版本中也是生码质量下滑的高频原因，需要专项优化。

D2C 还原：结构分析驱动实现策略

约定需求描述中存在 @D2C（d2c模块Id），会走到D2C还原链路。借助Imgcook D2C MCP可以将设计稿一键转换为 React 组件代码，早期实现中 Agent 获取到 D2C 结果后直接开始编码，忽略了一个关键问题：当前项目中可能已存在对应组件，此时应该“样式迁移”还是“结构重写”，Agent 完全靠猜，导致频繁选错策略。

对应的优化方式是：Agent在生码过程中先判断当前功能点是否已经存在，同时结合D2C的结果执行结构分析流程，再决定实现策略：

1. 提取 D2C 的 DOM 层级树，标注每层布局方式（flex 方向、嵌套层级、元素顺序）
2. 若存在现有组件，同步绘制现有组件的 DOM 层级树，与 D2C 逐层对比
3. 输出差异清单：明确标注哪些元素需要【删除】、【新增】、【移动】、【布局变更】
4. 确定实现策略：差异涉及布局重构 → 结构重写；差异仅为样式数值 → 样式迁移

API 接口：结构化解析 + 服务层 mock

约定需求描述中存在 **@API（接口Id，注册在业务功能树上）**，会走到API解析链路。借助 alex-code-knowledge MCP，可以获取到接口的完整定义（path、method、params、response），Agent 在拿到接口定义后需完成三件事：**类型生成、结构处理、接口数据mock：**

类型生成：根据 `params` 生成请求参数 interface，根据 `response.model` 生成响应 VO。需要注意 params/response 均为 JSON 字符串，使用前必须先解析；response 中的注释往往包含枚举说明（如 `status: 1|2|3, //1-草稿,2-审核中,3-生效`），需转为代码中的状态映射常量，而非硬编码数字。

结构处理：营销中后台接口遵循统一的响应结构，Agent 需按固定模式处理——检查 `success` 字段判断请求结果，从 `model` 字段提取业务数据，将 `msgInfo` 作为错误提示展示；分页场景下分页字段（`curPage`、`pageSize`、`totalCount`）统一在 `model` 中。

接口数据mock：需求中出现“接口未实现”时，会在服务函数内部直接生成 mock 数据并返回，同时用注释保留真实调用逻辑，供服务端实现后一键切换。

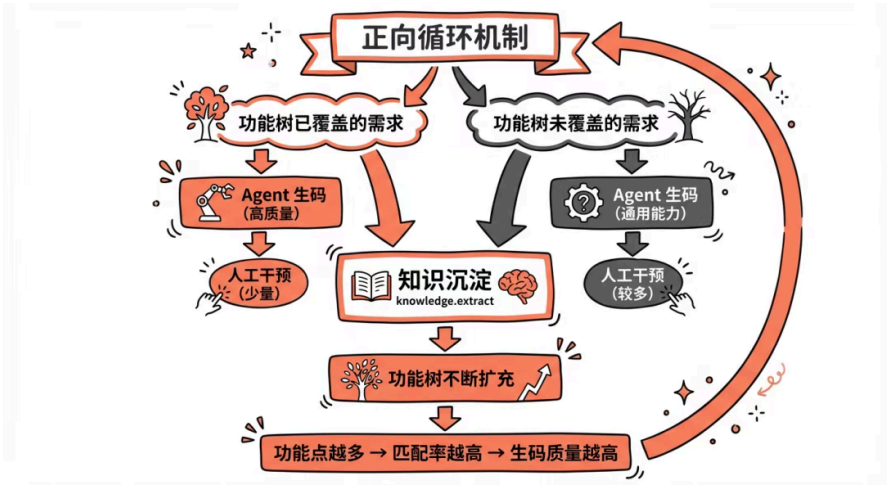
当需求输入中同时包含 **@D2C** 和 **@API** 时，两个 MCP 并行调用后以 API 的 response 为准定义 VO 类型，将 D2C Mock 数据的字段映射到 API 真实字段，避免 D2C Mock 数据结构混入生产代码。

- 知识沉淀的正向循环

功能树最初由人工梳理，随需求量增长，人工维护成本会成为瓶颈。为此，在工作流中引入了**知识自动沉淀**机制，让每次需求迭代的产出不只是代码，还会同步回流到功能树，每次迭代完成后，可一键触发功能树沉淀流程，基于本次需求AI生成的结果，以及人工干预的过程进行Skill新增或扩充，并自动挂靠在对应功能点上。



通过功能树沉淀能力，可建立起一套正向循环机制，功能树覆盖的需求会越来越多，人工需要的干预会越来越少：



- 落地效果

商家营销工具日常迭代：在已落地的 10+ 业务需求中，基本可实现平均 50% 以上的 AI 生码采纳率，部分需求类型下，采纳率可达80%以上。

整体来看，功能树覆盖的需求点，生码完成度可从原来的 40%~50% 提升至 80%~90%；未覆盖的需求点，Agent 仍可依靠自主研发能力完成基础实现，人工二次干预补齐剩余部分。随着功能树节点的不断扩充，整体覆盖率持续提升，形成可持续改善的提效飞轮。

05 总结与反思

FY26 S2，营销中后台的AI生码提效探索完成了一次系统性升级——从 S1 的“点状辅助”演进为覆盖需求交付全链路的“一体化工作流”。

在大量实践中，也沉淀出几条可复用的方法论：

- **给恰好够用的精确知识，而非更多知识：**上下文过载是生码质量下滑最常见的根因，知识的精确度比知识的完整度更重要。
- **确定性逻辑交还工程，不确定性决策留给 Agent：**将可枚举的路由、映射、格式处理收口到工程链路，让 Agent 专注于真正需要推理的创作任务，实现"合适的角色做合适的事"。
- **知识要形成正向循环，而非一次性投入：**通过功能树的自动沉淀机制，每次需求迭代的产出不只是代码，还会回流为下次生码的知识资产，构建可持续改善的提效飞轮。

当前方案仍有持续演进空间：功能树的覆盖率随业务迭代自然增长，分层研发指引也会随需求类型的丰富不断完善。从更长远的视角看，随着 AI 能力的持续升级和私域知识库的不断沉淀，营销中后台的AI生码路径将朝着更高自动化程度、更低人工干预成本的方向持续演进。

06 团队介绍

本文作者棣棠，来自淘天集团–营销前台技术团队。我们专注建设核心电商系统技术底座，直接支撑淘宝天猫的商业运转效率。团队致力于通过智能新方法优化消费者体验，对商家经营效率负责；我们将前沿AI知识与澎湃算力转化为实实在在的生产力，不断追求智能技术的上限，期望让科技进步真正造福万家灯火。

拓展阅读

3DXR技术 | 终端技术 | 音视频技术

服务端技术 | 技术质量 | 数据算法

大淘宝技术

大淘宝技术官方账号

967篇原创内容

公众号

阅读 1.2万

留言

写留言

还没有留言，来写首评